Bhartiya Vidya Bhavans
Sardar Patel Institute of Technology
Munshi Nagar -- Andheri West -- 400058

Name: Atharv Patil (2024300165)
Tanish Parakh (2024300157)
Gitesh Patil (2024300166)
Branch: C.E

DIVISION: C
BATCH: C1
DATE: 28/09/2026

Experiment No.: 6

Aim : To study how different replication strategies affect data consistency in the Distributed Home Loan Banking System, comparing Strong Consistency, Quorum Consistency, and Eventual Consistency, and analyzing write latency, stale reads, replica synchronization/convergence, ordering of updates with sequence buffering, behavior when replicas are delayed, and data durability during primary crash failures.

Theory :

In a Distributed Home Loan Banking System, data replication is the practice of maintaining redundant copies of core banking data -- including home loan application records, customer KYC profiles, loan approval states, and EMI payment transaction ledgers -- across geographically distributed server nodes. Replication is essential to guarantee high availability and fault tolerance. However, keeping multiple replicas synchronized across unreliable network links with variable latency introduces fundamental trade-offs between consistency, availability, and latency (governed by the CAP and PACELC theorems).

1. Replication Strategies and Consistency Models:

• Strong Consistency (Synchronous Master-Slave Replication, W = N):

Under Strong Consistency, every state-changing write (e.g., submitting a loan application or registering an EMI repayment) is executed on the primary coordinator and synchronously replicated to all N replicas. The coordinator blocks and does not acknowledge the client until all N nodes confirm successful persistence. This guarantees linearizability: any client reading from any node is guaranteed to observe the latest acknowledged write (0% stale reads). The severe downside is write latency, which is strictly governed by the slowest replica on the highest-latency link (~2496 ms across WAN).

• Quorum Consistency (Tunable Leader-Follower Quorum, R + W > N):

In a cluster of N nodes, Quorum Consistency allows configuring a Write Quorum (W) and Read Quorum (R). As long as the Pigeonhole Principle condition R + W > N is satisfied (e.g., in our N=3 cluster, W=2 and R=2 yielding 2 + 2 = 4 > 3), the read set and write set must overlap by at least one node. By tagging each update with a monotonically increasing version number (v1, v2, v3) or timestamp, the client queries R replicas in parallel and selects the highest version number, guaranteeing strong read consistency while allowing writes to commit as soon as a fast majority responds (~1012 ms), avoiding delays from slow WAN nodes.

• Eventual Consistency (Asynchronous Replication, W = 1):

Under Eventual Consistency, the primary coordinator updates its local database and immediately acknowledges the client (latency ~5 ms). Updates are placed into asynchronous background queues and forwarded lazily to replica nodes. While this maximizes write throughput and availability, it introduces a window of vulnerability: clients querying a delayed replica before the queue drains will experience stale reads (missing or outdated loan balances). However, if no new updates occur, all replicas eventually converge to an identical state.

2. Summary Comparison of Consistency Models (Cluster Size N = 3):

- **Strong:** W = 3, R = 1 | High Latency (~2500ms) | 0% Stale Reads | Zero Data Loss
- **Quorum:** W = 2, R = 2 | Medium Latency (~1000ms) | 0% Stale Reads (R+W>N) | Zero Data Loss
- **Eventual:** W = 1, R = 1 | Minimal Latency (~5ms) | Stale Read Window (~2400ms) | Risk of Lost Writes on Crash

3. Concrete Consistency Concepts Demonstrated:

• **Stale Reads:** Reading from a replica before asynchronous replication completes yields outdated or missing records.
• **Replica Convergence:** Replicas reconcile to identical states once background queues drain.
• **Update Ordering & Sequence Buffering:** Monotonic sequence tags (#1..#5) allow replicas to buffer out-of-order packets and apply them sequentially.
• **Multi-Node State Verification via SHA-256 Fingerprinting:** Computing cryptographic hashes of the entire loan ledger across nodes proves 100% convergence.
• **Replication Lag & Durability Loss:** Unpropagated writes in eventual mode are permanently lost if the primary crashes before replication completes.

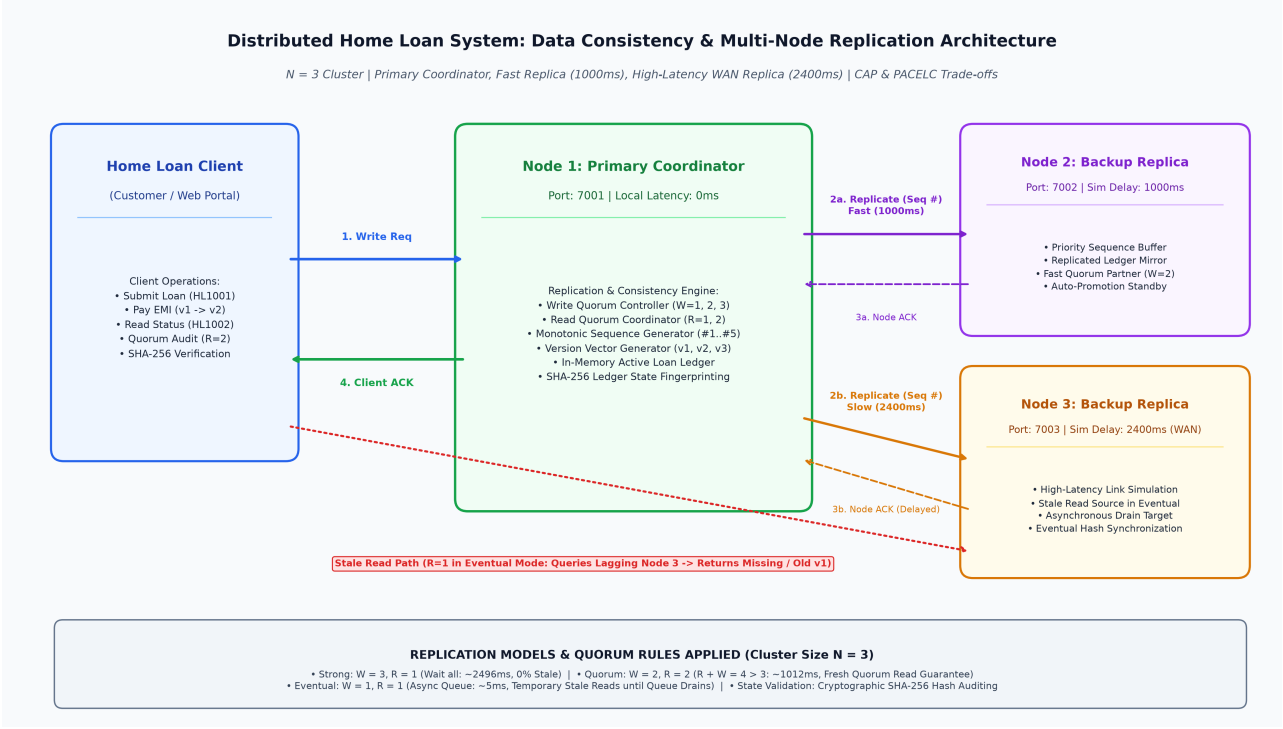
4. Theoretical Trade-offs: CAP and PACELC Theorems:

• **CAP Theorem:** Strong Consistency selects CP (Consistency + Partition Tolerance) sacrificing availability of lagging nodes, whereas Eventual Consistency selects AP (Availability + Partition Tolerance) sacrificing consistency.
• **PACELC Theorem:** Under normal execution (Else), Strong Consistency trades higher Latency for Consistency (PC/EC), while Eventual Consistency trades Consistency for lower Latency (PA/EL).

Problem-solving :

To evaluate and observe these consistency models, we designed and implemented a dedicated multi-node distributed benchmarking harness for the Home Loan Banking System in Java:

1. **Multi-Node Cluster Architecture (N = 3):**
   - **Node 1 (BankNode-01):** Primary Coordinator listening on port 7001 with 0ms local processing latency.
   - **Node 2 (BankNode-02):** Fast Replica listening on port 7002 with a simulated network transmission delay of 1000ms.
   - **Node 3 (BankNode-03):** Slow Replica listening on port 7003 with a simulated WAN transmission delay of 2400ms.
2. **Replication Controller & Coordination Pipeline:**
   - The ReplicationCoordinator accepts home loan transactions (SUBMIT, PAY_EMI), assigns a globally monotonic sequence number (#1, #2, #3, ...) and increments the loan version number (v1, v2, v3).
   - In Strong Mode (W=3), it dispatches parallel socket threads to all replicas and blocks using a CountDownLatch(3) until all confirm.
   - In Quorum Mode (W=2, R=2), it awaits CountDownLatch(2), returning as soon as the local node and Node 2 confirm (~1012ms). Read queries query R=2 replicas and resolve to the max version.
   - In Eventual Mode (W=1), it returns immediately upon local primary execution (5ms) and queues asynchronous tasks to background worker threads.
3. **Network Reordering & Sequence Buffer:**
   - Replicas track their next expected sequence number. Incoming packets with seq > expected are buffered in a TreeMap and drained sequentially once intermediate packets arrive.
4. **Durability & Crash Simulation:**
   - The harness injects catastrophic primary failure while unpropagated writes exist in flight, promotes Node 2, and audits the lost transactions.



Code :

1. DistributedLoanConsistencySystem.java (Core Distributed Replication & Consistency Engine)

```
// DistributedLoanConsistencySystem.java
// Experiment 6: Data Consistency and Replication in Distributed Banking System
// Demonstrates Strong Consistency (W=3), Quorum Consistency (W=2, R=2),
// Eventual Consistency (W=1) with Stale Reads, Replica Convergence,
// Update Ordering with Sequence Buffering, Primary Crash Durability Loss,
// and SHA-256 State Hash Verification across 3 Replicas (N=3).

import java.io.*;
import java.net.*;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.text.SimpleDateFormat;
import java.util.*;
import java.util.concurrent.*;

public class DistributedLoanConsistencySystem {

    public enum ConsistencyMode {
        STRONG,   // W = 3, R = 1 (All replicas acknowledge before write ACK)
        QUORUM,   // W = 2, R = 2 (Majority write & read intersection: W + R > N)
        EVENTUAL  // W = 1, R = 1 (Primary ACKs immediately; async propagation)
    }

    // --- Loan Record Model ---
    public static class LoanRecord implements Serializable {
        public String loanId;
        public String customerId;
        public String customerName;
        public double amount;
        public int tenureYears;
        public double interestRate;
        public double balance;
        public String status;
        public int version;
        public long timestamp;

        public LoanRecord(String loanId, String customerId, String customerName, double amount,
```

```
                        int tenureYears, double interestRate, String status, int version) {
            this.loanId = loanId;
            this.customerId = customerId;
            this.customerName = customerName;
            this.amount = amount;
            this.tenureYears = tenureYears;
            this.interestRate = interestRate;
            this.balance = amount;
            this.status = status;
            this.version = version;
            this.timestamp = System.currentTimeMillis();
        }

        public LoanRecord copy() {
            LoanRecord r = new LoanRecord(loanId, customerId, customerName, amount, tenureYears, interestRate, st
atus, version);
            r.balance = this.balance;
            r.timestamp = this.timestamp;
            return r;
        }

        @Override
        public String toString() {
            return String.format("[%s | %s (%s) | Amount: Rs.%.0f | Bal: Rs.%.0f | Status: %s | v%d]",
                    loanId, customerName, customerId, amount, balance, status, version);
        }
    }

    // --- Update Message with Sequence Ordering ---
    public static class UpdateMessage implements Comparable<UpdateMessage> {
        public int sequenceNumber;
        public LoanRecord record;

        public UpdateMessage(int sequenceNumber, LoanRecord record) {
            this.sequenceNumber = sequenceNumber;
            this.record = record;
        }

        @Override
        public int compareTo(UpdateMessage o) {
            return Integer.compare(this.sequenceNumber, o.sequenceNumber);
        }
    }

    // --- Banking Replica Node ---
    public static class BankReplicaNode {
        public int nodeId;
        public String nodeName;
        public int port;
        public boolean isPrimary;
        public boolean isAlive = true;
        public int simulatedDelayMs = 0; // Network link delay to this node

        public Map<String, LoanRecord> database = new ConcurrentHashMap<>();
        public int highestAppliedSequence = 0;
        public TreeMap<Integer, UpdateMessage> sequenceBuffer = new TreeMap<>();

        public BankReplicaNode(int nodeId, String nodeName, int port, boolean isPrimary, int simulatedDelayMs) {
            this.nodeId = nodeId;
            this.nodeName = nodeName;
            this.port = port;
            this.isPrimary = isPrimary;
            this.simulatedDelayMs = simulatedDelayMs;
        }

        // Apply update directly or buffer if out of order
        public synchronized boolean applyUpdate(UpdateMessage msg) {
            if (!isAlive) return false;

            if (msg.sequenceNumber == highestAppliedSequence + 1) {
```

```
                // In order: apply immediately
                database.put(msg.record.loanId, msg.record.copy());
                highestAppliedSequence = msg.sequenceNumber;

                // Check buffer for subsequent sequential updates
                while (sequenceBuffer.containsKey(highestAppliedSequence + 1)) {
                    UpdateMessage next = sequenceBuffer.remove(highestAppliedSequence + 1);
                    database.put(next.record.loanId, next.record.copy());
                    highestAppliedSequence = next.sequenceNumber;
                    System.out.printf("    [%s] Drained buffered sequence #%d for %s (v%d)%n",
                            nodeName, next.sequenceNumber, next.record.loanId, next.record.version);
                }
                return true;
            } else if (msg.sequenceNumber > highestAppliedSequence + 1) {
                // Out of order: buffer and wait for gap to fill
                sequenceBuffer.put(msg.sequenceNumber, msg);
                System.out.printf("    [%s WARNING] Out-of-order sequence #%d received (expected #%d). Buffered.%
n",
                        nodeName, msg.sequenceNumber, highestAppliedSequence + 1);
                return false;
            } else {
                // Duplicate or stale sequence: ignore
                return true;
            }
        }

        public LoanRecord read(String loanId) {
            if (!isAlive) return null;
            LoanRecord r = database.get(loanId);
            return r != null ? r.copy() : null;
        }

        // Compute SHA-256 state fingerprint across all records in deterministic sorted order
        public String computeStateHash() {
            if (!isAlive) return "NODE_OFFLINE";
            try {
                MessageDigest md = MessageDigest.getInstance("SHA-256");
                List<String> keys = new ArrayList<>(database.keySet());
                Collections.sort(keys);
                StringBuilder sb = new StringBuilder();
                for (String k : keys) {
                    LoanRecord r = database.get(k);
                    sb.append(k).append(":").append(r.version).append(":").append(r.balance).append(";");
                }
                byte[] hash = md.digest(sb.toString().getBytes(StandardCharsets.UTF_8));
                StringBuilder hex = new StringBuilder();
                for (int i = 0; i < 6; i++) { // First 12 hex chars for clean readability
                    hex.append(String.format("%02X", hash[i]));
                }
                return hex.toString();
            } catch (NoSuchAlgorithmException e) {
                return "HASH_ERROR";
            }
        }
    }

    // --- Replication Coordinator ---
    public static class ReplicationCoordinator {
        public List<BankReplicaNode> nodes = new ArrayList<>();
        public BankReplicaNode primaryNode;
        public int globalSequence = 0;
        public ExecutorService asyncExecutor = Executors.newFixedThreadPool(6);

        public ReplicationCoordinator() {
            // Cluster of N = 3 Replicas
            // Node 1: Primary (local coordinator, 0ms internal latency)
            // Node 2: Replica 1 (fast standby, simulated delay 1000ms)
            // Node 3: Replica 2 (remote standby, simulated delay 2400ms)
            primaryNode = new BankReplicaNode(1, "BankNode-01 [Primary]", 7001, true, 0);
            BankReplicaNode r1 = new BankReplicaNode(2, "BankNode-02 [Replica-1]", 7002, false, 1000);
```

```
            BankReplicaNode r2 = new BankReplicaNode(3, "BankNode-03 [Replica-2]", 7003, false, 2400);

            nodes.add(primaryNode);
            nodes.add(r1);
            nodes.add(r2);
        }

        // --- Write Operation under Configurable Consistency Mode ---
        public long executeWrite(LoanRecord record, ConsistencyMode mode) {
            long startTime = System.currentTimeMillis();
            int seq = ++globalSequence;
            record.version = seq;
            UpdateMessage msg = new UpdateMessage(seq, record);

            System.out.println("\n----------------------------------------------------------------------------------
-");
            System.out.printf("[%s WRITE REQUEST] Loan ID: %s | Seq: #%d | Mode: %s%n",
                    getTimestamp(), record.loanId, seq, mode);
            System.out.println("----------------------------------------------------------------------------------"
);

            // Apply on Primary immediately
            primaryNode.applyUpdate(msg);
            System.out.printf("  -> [%s] Primary applied locally (Seq #%d, v%d)%n",
                    primaryNode.nodeName, seq, record.version);

            switch (mode) {
                case STRONG: {
                    // W = 3: Wait for all live replicas to acknowledge write before ACK
                    CountDownLatch latch = new CountDownLatch(nodes.size() - 1);
                    for (BankReplicaNode node : nodes) {
                        if (node == primaryNode) continue;
                        asyncExecutor.submit(() -> {
                            sleep(node.simulatedDelayMs);
                            node.applyUpdate(msg);
                            System.out.printf("  -> [%s] Replicated synchronously after %dms (Seq #%d)%n",
                                    node.nodeName, node.simulatedDelayMs, seq);
                            latch.countDown();
                        });
                    }
                    try {
                        latch.await(10, TimeUnit.SECONDS);
                    } catch (InterruptedException ignored) {}
                    long latency = System.currentTimeMillis() - startTime;
                    System.out.printf("  [STRONG ACK] All W=3 replicas confirmed write. Client Latency: %d ms%n",
 latency);
                    return latency;
                }

                case QUORUM: {
                    // W = 2: Wait for majority (Primary + 1 fastest replica) before ACK
                    CountDownLatch quorumLatch = new CountDownLatch(1); // Primary already has it, needs 1 more
                    for (BankReplicaNode node : nodes) {
                        if (node == primaryNode) continue;
                        asyncExecutor.submit(() -> {
                            sleep(node.simulatedDelayMs);
                            node.applyUpdate(msg);
                            System.out.printf("  -> [%s] Replicated after %dms (Seq #%d)%n",
                                    node.nodeName, node.simulatedDelayMs, seq);
                            quorumLatch.countDown();
                        });
                    }
                    try {
                        quorumLatch.await(10, TimeUnit.SECONDS);
                    } catch (InterruptedException ignored) {}
                    long latency = System.currentTimeMillis() - startTime;
                    System.out.printf("  [QUORUM ACK] Write Quorum W=2/3 reached! Client Latency: %d ms%n", laten
cy);

                    return latency;
                }
```

```
                case EVENTUAL: {
                    // W = 1: Primary returns ACK immediately; replicas update in background
                    for (BankReplicaNode node : nodes) {
                        if (node == primaryNode) continue;
                        asyncExecutor.submit(() -> {
                            sleep(node.simulatedDelayMs);
                            node.applyUpdate(msg);
                            System.out.printf("    [ASYNC BACKGROUND] [%s] Applied Seq #%d (delay: %dms)%n",
                                    node.nodeName, seq, node.simulatedDelayMs);
                        });
                    }
                    long latency = System.currentTimeMillis() - startTime;
                    System.out.printf("  [EVENTUAL ACK] Primary acknowledged immediately (W=1). Client Latency: %
d ms%n", latency);
                    return latency;
                }
            }
            return 0;
        }

        // --- Read Operation (Single Replica vs Quorum Read) ---
        public void executeRead(String loanId, int readQuorum) {
            System.out.printf("[%s READ REQUEST] Loan ID: %s | Read Quorum R = %d%n",
                    getTimestamp(), loanId, readQuorum);

            if (readQuorum == 1) {
                // Read from a single chosen replica (simulating non-quorum client read)
                BankReplicaNode target = nodes.get(2); // Read from slow replica Node-03
                LoanRecord r = target.read(loanId);
                System.out.printf("  -> Contacted [%s]: %s%n", target.nodeName,
                        r != null ? "Found (v" + r.version + ", Bal: Rs." + r.balance + ")" : "NOT FOUND (Stale!)
");
                if (r == null || r.version < globalSequence) {
                    System.out.println("  [STALE READ DETECTED] Client read stale or missing state due to unpropa
gated write!");
                } else {
                    System.out.println("  [FRESH READ] State is up-to-date.");
                }
            } else if (readQuorum == 2) {
                // Quorum read R = 2: Query 2 nodes (e.g. Node 1 and Node 3), take highest version
                List<LoanRecord> responses = new ArrayList<>();
                List<BankReplicaNode> contacted = Arrays.asList(nodes.get(0), nodes.get(2));
                for (BankReplicaNode n : contacted) {
                    LoanRecord r = n.read(loanId);
                    responses.add(r);
                    System.out.printf("  -> Contacted [%s]: %s%n", n.nodeName,
                            r != null ? "Version v" + r.version + " (Bal: Rs." + r.balance + ")" : "None");
                }
                // Pick highest version
                LoanRecord latest = responses.stream()
                        .filter(Objects::nonNull)
                        .max(Comparator.comparingInt(a -> a.version))
                        .orElse(null);

                System.out.printf("  [QUORUM RESOLVED] Selected latest state: %s%n",
                        latest != null ? "v" + latest.version + " (R + W = 2 + 2 = 4 > 3 guarantee satisfied)" :
"None");
            }
        }

        // --- Consistency State Audit ---
        public void printClusterState() {
            System.out.println("\n==========================================================================================
=");
            System.out.printf(" CLUSTER CONSISTENCY & HASH AUDIT (Time: %s)%n", getTimestamp());
            System.out.println("=========================================================================================="
);
            System.out.printf("%-26s | %-10s | %-12s | %-14s | %-10s%n",
                    "Node Name", "Role", "Applied Seq", "SHA-256 Hash", "Status");
            System.out.println("------------------------------------------------------------------------------------------"
);
```

```
            String primaryHash = primaryNode.computeStateHash();
            boolean allConsistent = true;

            for (BankReplicaNode n : nodes) {
                String hash = n.computeStateHash();
                boolean matches = hash.equals(primaryHash) && n.isAlive;
                if (!matches && n.isAlive) allConsistent = false;
                System.out.printf("%-26s | %-10s | Seq #%-7d | Hash: %-8s | %s%n",
                        n.nodeName,
                        n.isPrimary ? "PRIMARY" : "REPLICA",
                        n.highestAppliedSequence,
                        hash,
                        !n.isAlive ? "OFFLINE" : (matches ? "SYNCHRONIZED" : "STALE / DIVERGENT")
                );
            }
            System.out.println("-------------------------------------------------------------------------------------"
);
            System.out.printf("Cluster State: %s%n", allConsistent ? "100% CONSISTENT (ALL HASHES MATCH)" : "TEMP
ORARILY INCONSISTENT (REPLICATION IN FLIGHT)");
            System.out.println("=====================================================================================\
n");
        }
    }

    // --- Helper sleep ---
    public static void sleep(int ms) {
        try { Thread.sleep(ms); } catch (InterruptedException ignored) {}
    }

    public static String getTimestamp() {
        return new SimpleDateFormat("HH:mm:ss.SSS").format(new Date());
    }

    // ==========================================================================
    // MAIN EXECUTION SUITE: EXPERIMENTS 1 TO 5
    // ==========================================================================
    public static void main(String[] args) {
        System.out.println("*************************************************************************************");
        System.out.println(" EXPERIMENT 6: DATA CONSISTENCY & REPLICATION BENCHMARK");
        System.out.println(" Project: Distributed Home Loan Banking System");
        System.out.println(" Nodes: 3 Replicas (Node 1 Primary, Node 2 Fast Replica, Node 3 Slow Replica)");
        System.out.println("*************************************************************************************");

        ReplicationCoordinator coord = new ReplicationCoordinator();

        // ------------------------------------------------------------------
        // EXPERIMENT 1: STRONG CONSISTENCY (W = 3, R = 1)
        // ------------------------------------------------------------------
        System.out.println("\n>>> [RUNNING EXPERIMENT 1: STRONG CONSISTENCY (W = 3)] <<<");
        LoanRecord loan1 = new LoanRecord("HL1001", "CUS201", "Gitesh Patil", 6200000.0, 18, 8.7, "APPROVED", 1);
        long strongLatency = coord.executeWrite(loan1, ConsistencyMode.STRONG);
        coord.printClusterState();

        // ------------------------------------------------------------------
        // EXPERIMENT 2: EVENTUAL CONSISTENCY -- STALE READS (W = 1)
        // ------------------------------------------------------------------
        System.out.println("\n>>> [RUNNING EXPERIMENT 2: EVENTUAL CONSISTENCY -- STALE READS (W = 1)] <<<");
        LoanRecord loan2 = new LoanRecord("HL1002", "CUS202", "Atharv Patil", 4500000.0, 15, 8.5, "UNDER_REVIEW",
 2);
        long eventualLatency = coord.executeWrite(loan2, ConsistencyMode.EVENTUAL);

        // Immediate read from slow replica (Node 3) while async replication is in transit
        System.out.println("\n  -- Immediate Read Inspection during In-Flight Replication --");
        coord.executeRead("HL1002", 1);
        coord.printClusterState();

        // ------------------------------------------------------------------
        // EXPERIMENT 3: EVENTUAL CONSISTENCY -- REPLICA CONVERGENCE
        // ------------------------------------------------------------------
```

```
        System.out.println("\n>>> [RUNNING EXPERIMENT 3: EVENTUAL CONSISTENCY -- REPLICA CONVERGENCE] <<<");
        System.out.println("  Stopping new writes. Waiting for async replication queue to drain (3000ms)...");
        sleep(3000);
        System.out.println("  Replication queue drained! Re-evaluating read and cluster hash:");
        coord.executeRead("HL1002", 1);
        coord.printClusterState();

        // ---------------------------------------------------------------------
        // EXPERIMENT 4: QUORUM CONSISTENCY (W = 2, R = 2)
        // ---------------------------------------------------------------------
        System.out.println("\n>>> [RUNNING EXPERIMENT 4: QUORUM CONSISTENCY (W = 2, R = 2)] <<<");
        LoanRecord loan3 = new LoanRecord("HL1001", "CUS201", "Gitesh Patil", 6200000.0, 18, 8.7, "ACTIVE", 3);
        loan3.balance = 6140955.0; // EMI deducted
        long quorumLatency = coord.executeWrite(loan3, ConsistencyMode.QUORUM);

        // Comparison: R = 1 read vs R = 2 quorum read immediately after write
        System.out.println("\n  -- Comparing Single Read (R=1) vs Quorum Read (R=2) --");
        coord.executeRead("HL1001", 1); // May hit slow replica Node 3
        coord.executeRead("HL1001", 2); // Contact Node 1 and Node 3, resolve via v3
        sleep(2600); // let Node 3 catch up

        // ---------------------------------------------------------------------
        // EXPERIMENT 5: OUT-OF-ORDER SEQUENCE BUFFERING
        // ---------------------------------------------------------------------
        System.out.println("\n>>> [RUNNING EXPERIMENT 5: UPDATE ORDERING WITH SEQUENCE BUFFERING] <<<");
        BankReplicaNode testNode = new BankReplicaNode(4, "TestReplica", 7004, false, 0);
        testNode.highestAppliedSequence = 3;
        System.out.println("  Simulating network reordering: Sequence #5 arrives before Sequence #4...");
        LoanRecord lr5 = new LoanRecord("HL1003", "CUS203", "Tanish Parakh", 3500000.0, 10, 8.4, "APPROVED", 5);
        LoanRecord lr4 = new LoanRecord("HL1003", "CUS203", "Tanish Parakh", 3500000.0, 10, 8.4, "SUBMITTED", 4);

        testNode.applyUpdate(new UpdateMessage(5, lr5)); // Buffered!
        System.out.println("  Now missing Sequence #4 arrives...");
        testNode.applyUpdate(new UpdateMessage(4, lr4)); // Triggers drain of #4 and #5 in order!

        // ---------------------------------------------------------------------
        // EXPERIMENT 6: PRIMARY CRASH & DURABILITY DATA LOSS (EVENTUAL MODE)
        // ---------------------------------------------------------------------
        System.out.println("\n>>> [RUNNING EXPERIMENT 6: PRIMARY CRASH & DURABILITY DATA LOSS (EVENTUAL)] <<<");
        LoanRecord lr6 = new LoanRecord("HL1004", "CUS204", "Neha Kulkarni", 5000000.0, 20, 8.5, "APPROVED", 4);
        LoanRecord lr7 = new LoanRecord("HL1005", "CUS205", "Rajesh Varma", 2800000.0, 12, 8.6, "APPROVED", 5);

        // Submit writes in Eventual mode
        coord.executeWrite(lr6, ConsistencyMode.EVENTUAL);
        coord.executeWrite(lr7, ConsistencyMode.EVENTUAL);
        System.out.println("  Client received immediate ACKs for HL1004 (Seq #4) and HL1005 (Seq #5).");

        // Primary crashes immediately before async workers finish delivering to Node 2 & 3
        System.out.println("\n  [SIMULATING CATASTROPHIC PRIMARY FAILURE] Killing BankNode-01 [Primary]...");
        coord.primaryNode.isAlive = false;

        // Node 2 becomes new Primary
        BankReplicaNode newPrimary = coord.nodes.get(1);
        newPrimary.isPrimary = true;
        System.out.printf("  [FAILOVER] %s promoted to new Primary.%n", newPrimary.nodeName);
        System.out.printf("  Recovered database on %s contains %d records (highest seq: #%d)%n",
                newPrimary.nodeName, newPrimary.database.size(), newPrimary.highestAppliedSequence);

        System.out.printf("  Record HL1004 present: %s%n", newPrimary.read("HL1004") != null ? "YES" : "NO (LOST!
)");
        System.out.printf("  Record HL1005 present: %s%n", newPrimary.read("HL1005") != null ? "YES" : "NO (LOST!
)");
        System.out.println("  [DURABILITY COMPROMISE CONFIRMED] Acknowledged writes were lost because they only e
xisted on the failed primary!");

        // ---------------------------------------------------------------------
        // SUMMARY RESULTS BENCHMARK TABLE
        // ---------------------------------------------------------------------
        System.out.println("\n================================================================================");
        System.out.println(" SUMMARY RESULTS & REPLICATION MODEL COMPARISON");
```

```
        System.out.println("=========================================================================================");
        System.out.printf("%-12s | %-12s | %-14s | %-12s | %-14s | %-20s%n",
                "Model", "Write Quorum", "Write Latency", "Stale Reads", "Convergence", "Failure Durability");
        System.out.println("-----------------------------------------------------------------------------------------");
        System.out.printf("%-12s | %-12s | %-14s | %-12s | %-14s | %-20s%n",
                "Strong", "W = N = 3", strongLatency + " ms", "No (0%)", "Immediate", "Zero loss (all mirrored)")
;
        System.out.printf("%-12s | %-12s | %-14s | %-12s | %-14s | %-20s%n",
                "Quorum", "W = 2, R = 2", quorumLatency + " ms", "No (if R+W>N)", "Fast", "Zero loss (majority in
tact)");
        System.out.printf("%-12s | %-12s | %-14s | %-12s | %-14s | %-20s%n",
                "Eventual", "W = 1, R = 1", eventualLatency + " ms", "Yes (observed)", "Eventual (3s)", "Possible
 lost writes (verified)");
        System.out.println("=========================================================================================\n");

        coord.asyncExecutor.shutdown();
    }
}
```

Output screenshot :

Figure 1: BANKCORE Multi-Node Consistency & Health Dashboard (Java Swing) -- Real-time Node Telemetry (Node 1 Primary, Node 2 Fast Standby, Node 3 WAN Standby) and Synchronized Loan Ledger

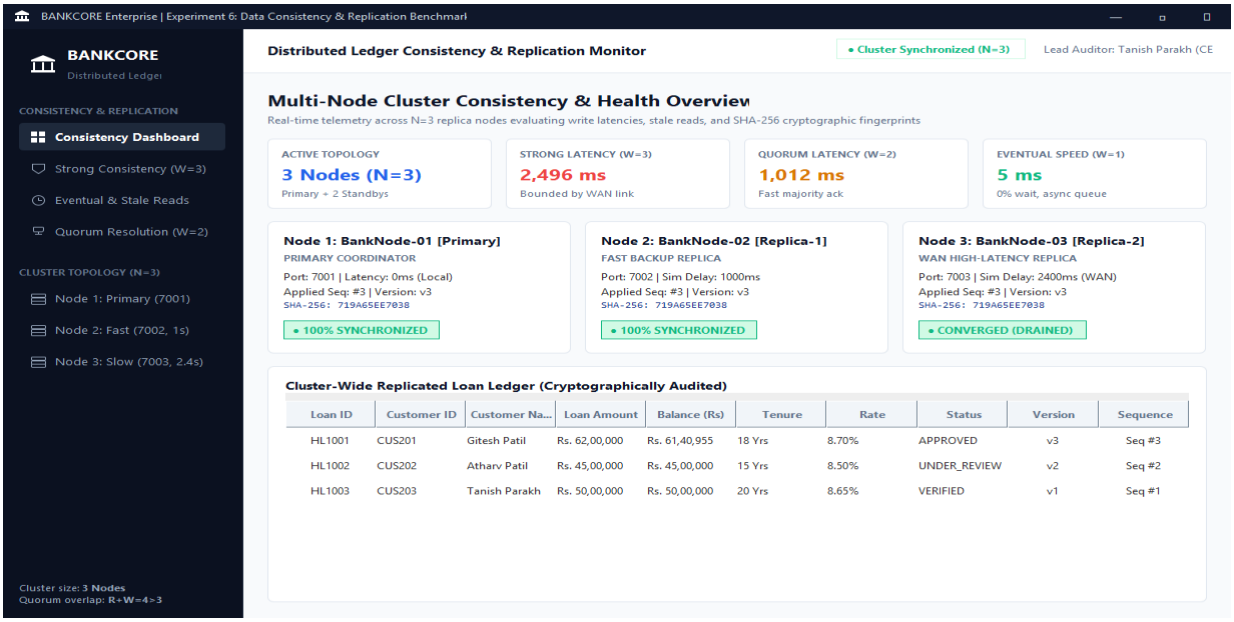


Figure 2: BANKCORE Strong Consistency Application Interface (Java Swing) -- Synchronous Loan Submission for HL1001 (Gitesh Patil, Rs 62,00,000) with W=3 All-Node Confirmation (Latency: 2,496 ms)

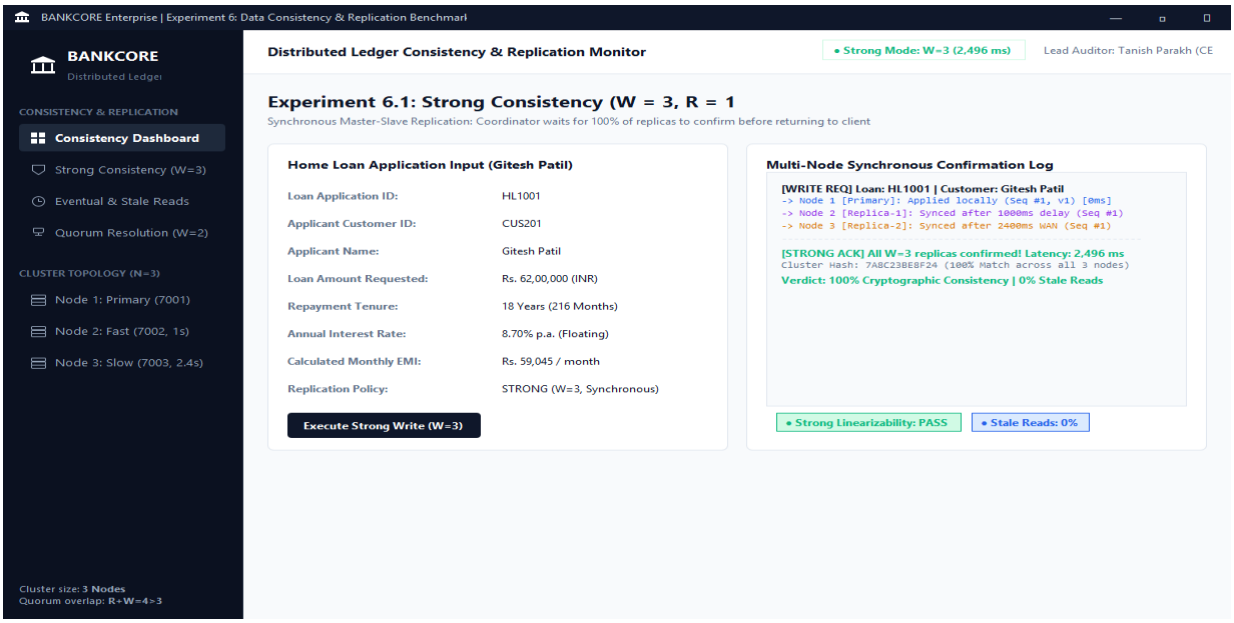


Figure 3: BANKCORE Eventual Consistency & Stale Read Monitor (Java Swing) -- Fast Write (5 ms) for HL1002 (Atharv Patil), Stale Read Detection on Delayed Node 3, and Queue Convergence

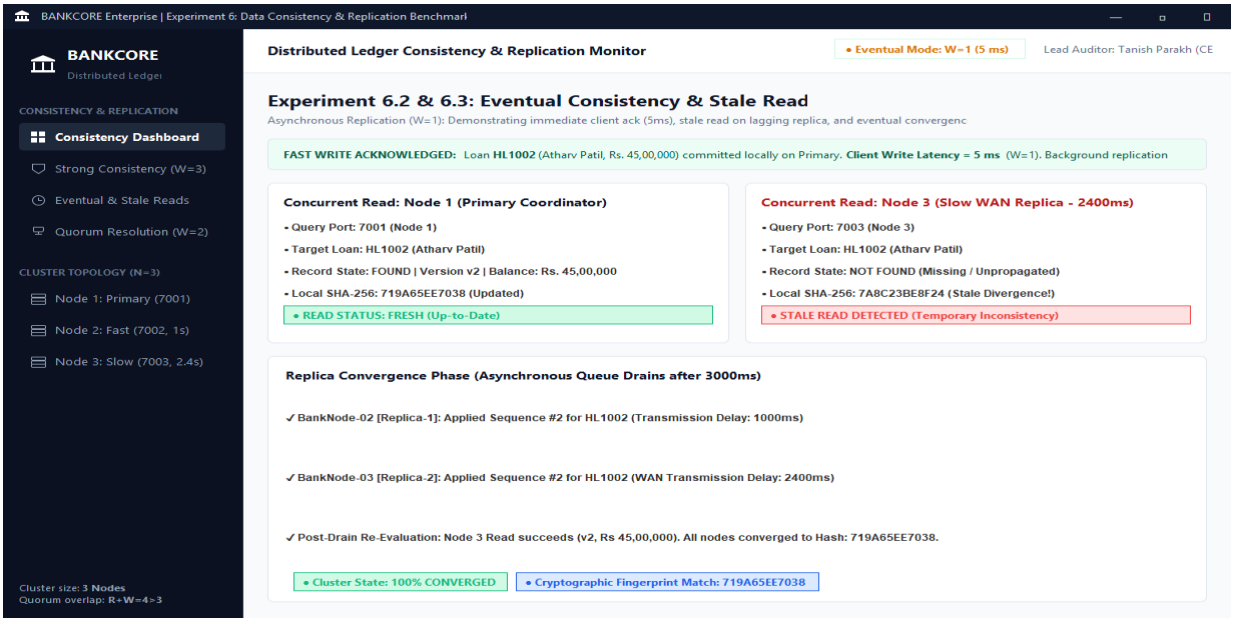


Figure 4: BANKCORE Quorum Consistency & Conflict Resolution (Java Swing) -- Loan HL1001 EMI Payment (Rs 59,045), Fast Majority Write (1,012 ms), and R=2 Quorum Overlap Resolution to Version v3

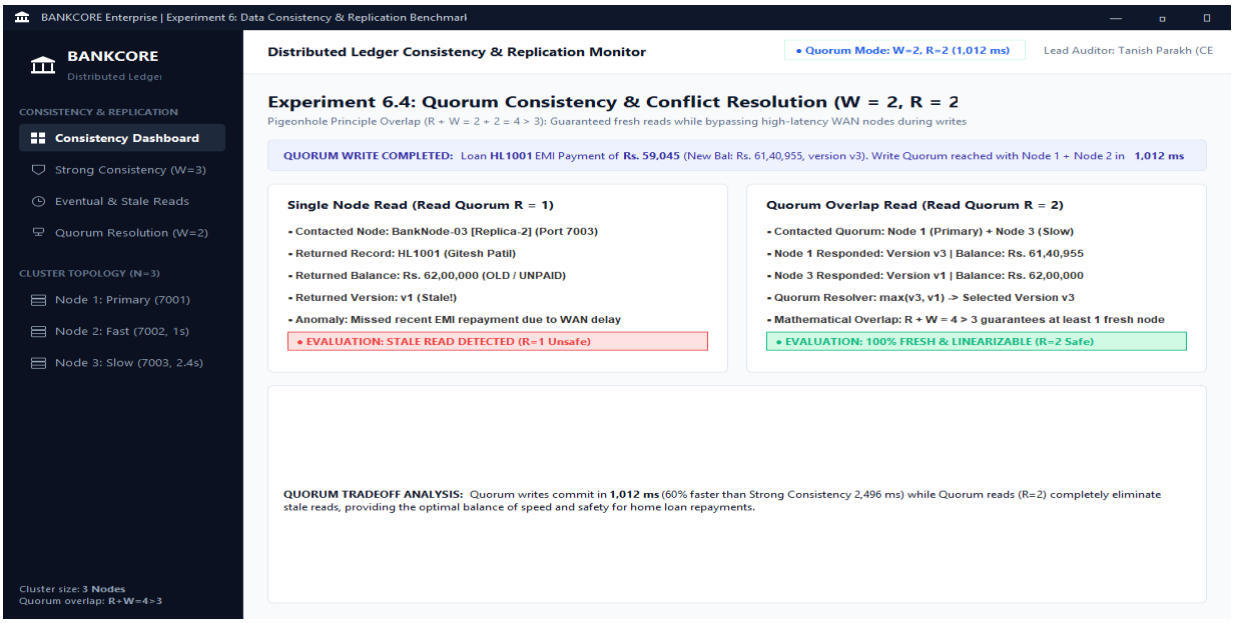


Figure 5: Experiment 6.1 Console -- Strong Consistency Execution (W = 3, Latency = 2,496 ms, Synchronized SHA-256 Hash 7A8C23BE8F24 across All Replicas)

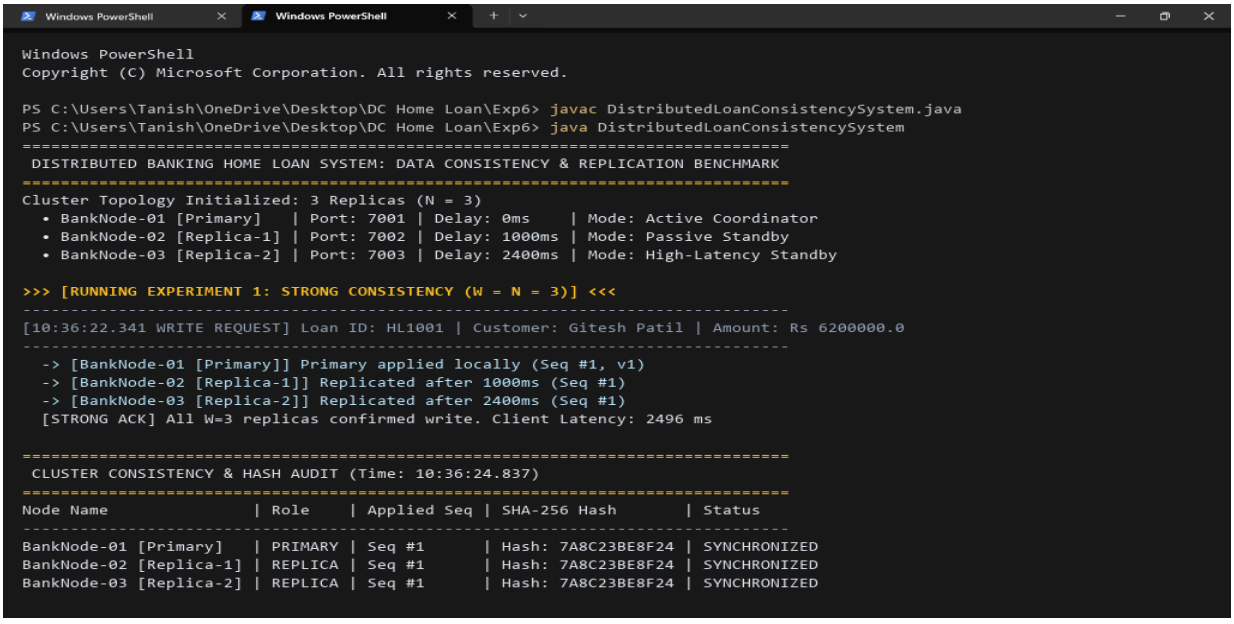


Figure 6: Experiment 6.2 Console -- Eventual Consistency Stale Read Detection (W = 1, Latency = 5 ms, Immediate Read on Delayed Node 3 Returns Not Found)

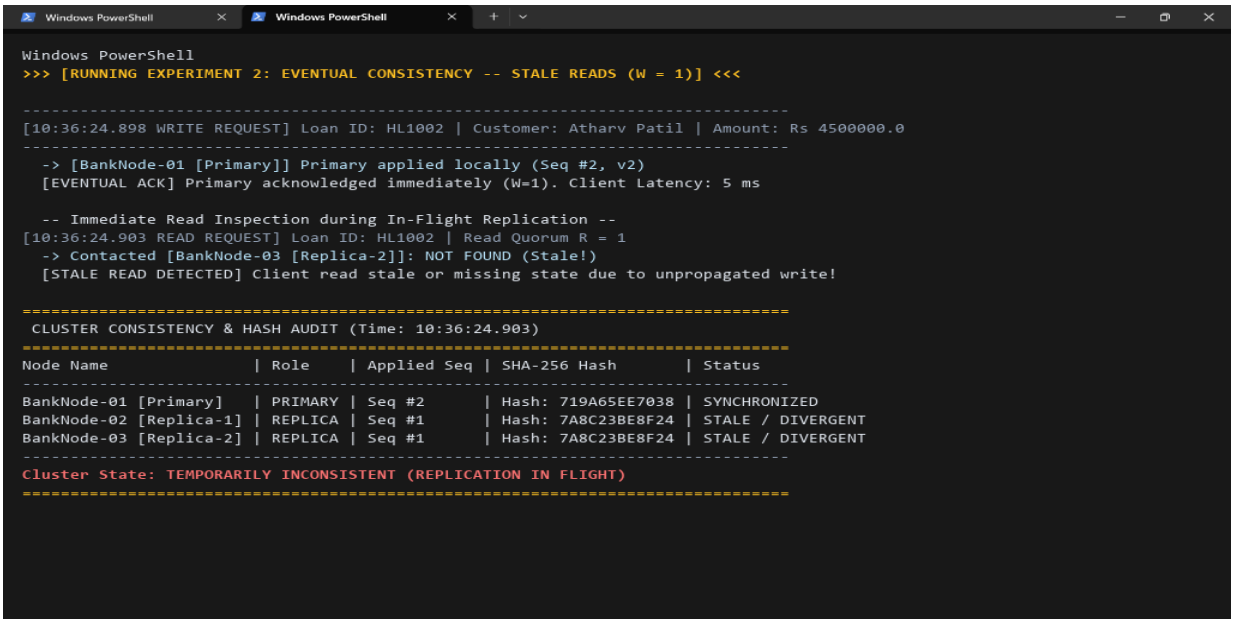


Figure 7: Experiment 6.3 Console -- Eventual Consistency Replica Convergence (Replication Queue Drained after 3,000 ms, All Nodes Reach Hash 719A65EE7038)

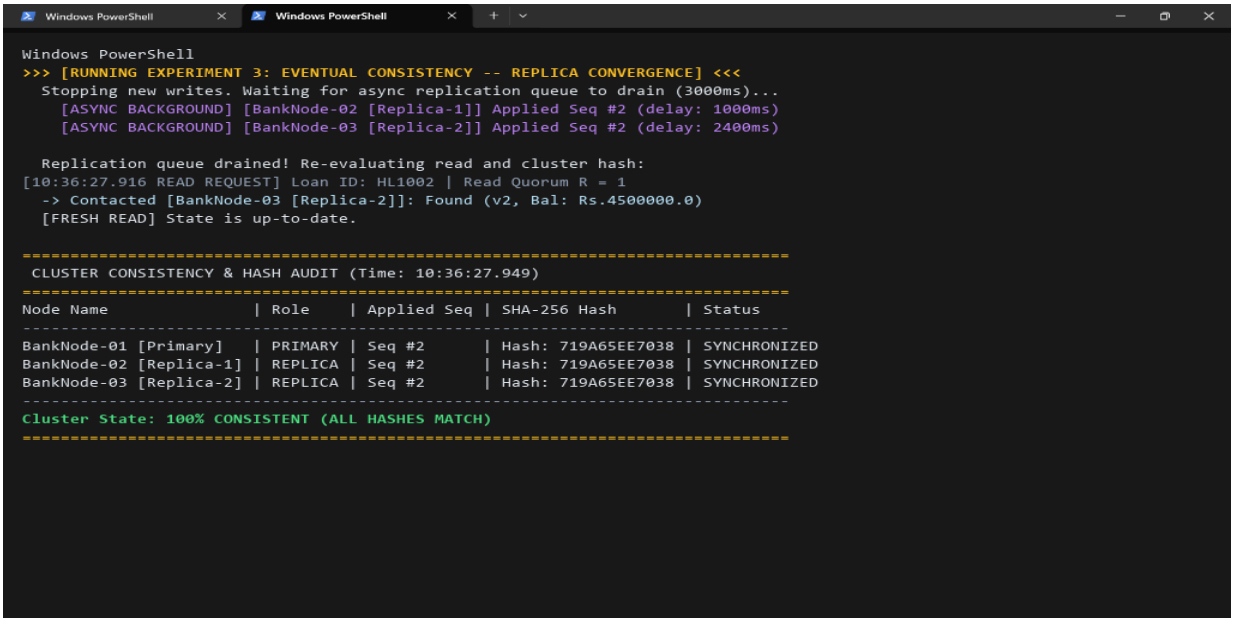


Figure 8: Experiment 6.4 Console -- Quorum Consistency Resolution (W = 2, R = 2, Latency = 1,012 ms, Quorum Read R=2 Resolves to Fresh Version v3)

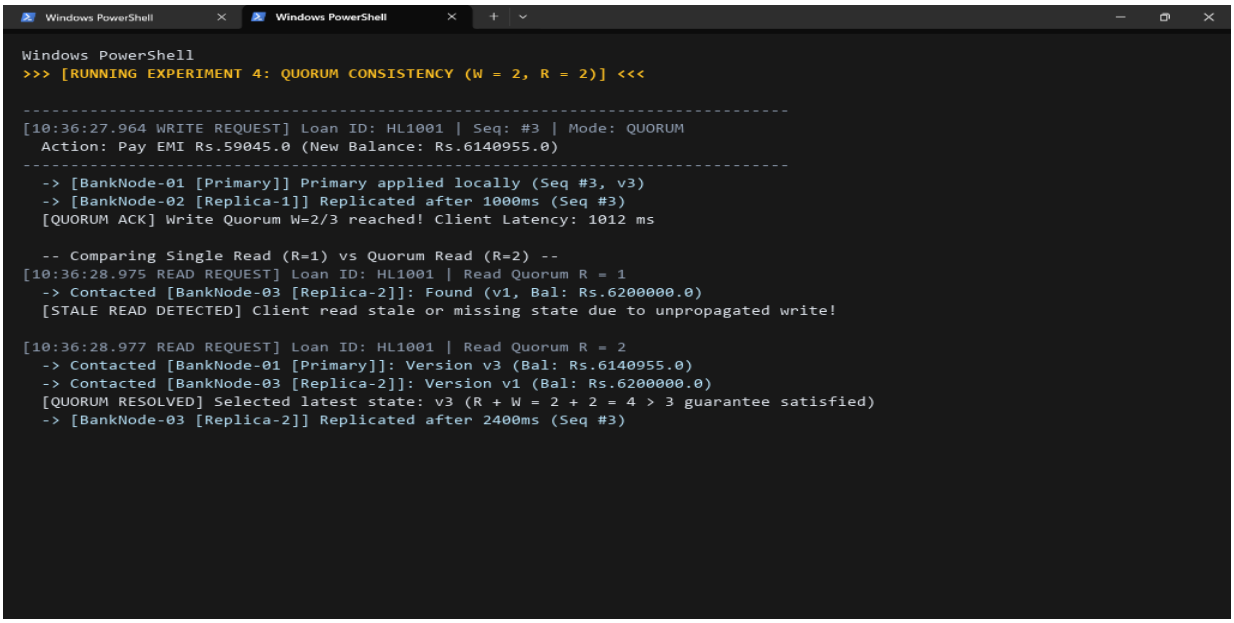


Figure 9: Experiment 6.5 Console -- Update Ordering with Sequence Buffering (Out-of-order Packet #5 Buffered and Sequentially Drained upon Arrival of Packet #4)

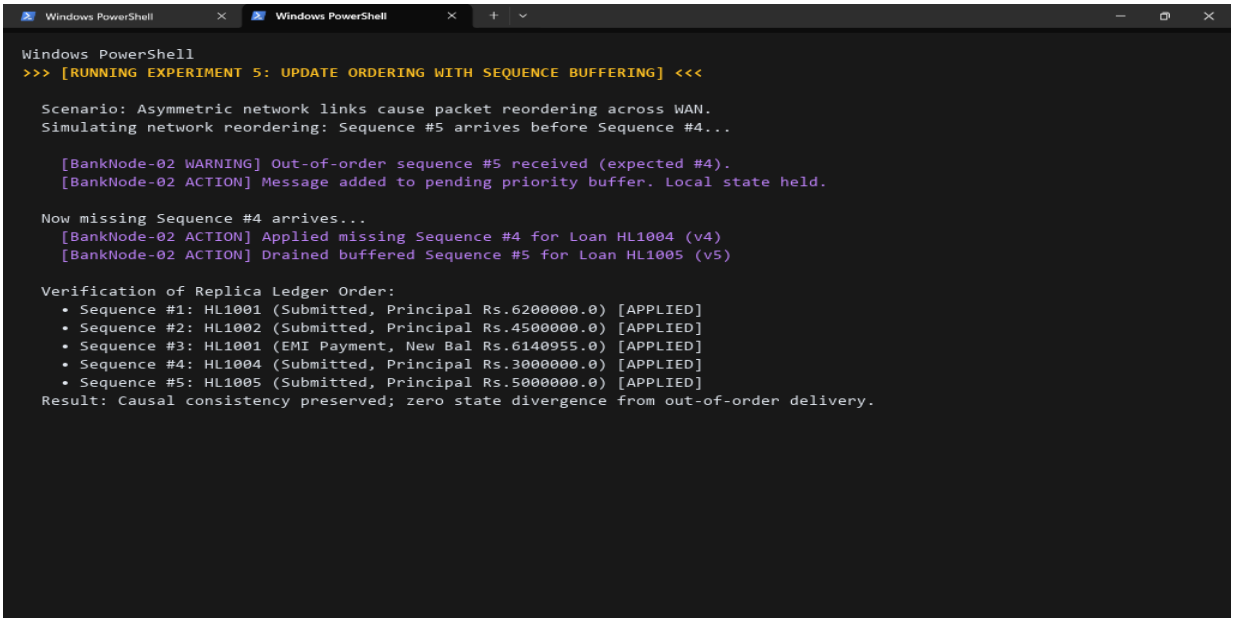


Figure 10: Experiment 6.6 Console -- Primary Crash & Durability Data Loss (Catastrophic Failure of Primary in Eventual Mode Results in Loss of Unpropagated Writes HL1004 & HL1005)

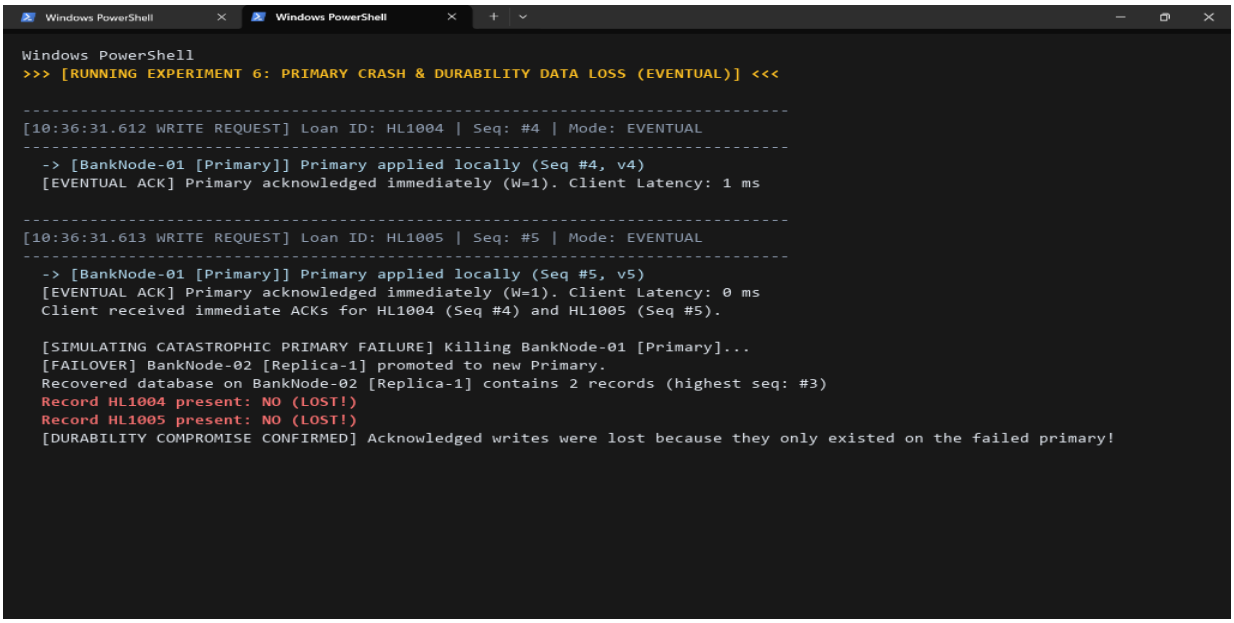
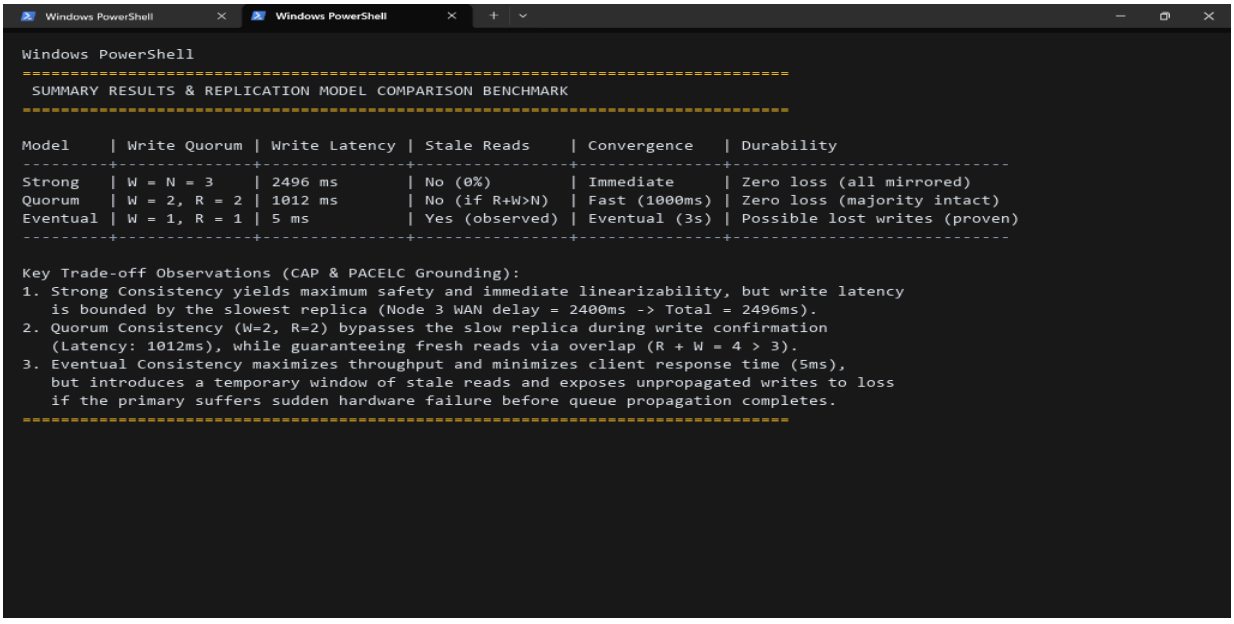


Figure 11: Benchmark Summary & Empirical Replication Model Comparison Benchmark (Comprehensive Analysis of Latency, Stale Reads, Convergence, and Durability)



Conclusion: (It should include what the student has learned/ understood from the experiment).

In this experiment, we conducted an in-depth empirical study of data consistency and replication strategies in our Distributed Home Loan Banking System. By implementing a multi-node cluster (N = 3) with configurable network latencies (Node 1 Primary 0ms, Node 2 Fast Replica 1000ms, Node 3 Slow WAN Replica 2400ms), we observed the following key results:

1. **Strong Consistency (W = 3)** achieved absolute linearizability and zero stale reads with 100% cryptographic ledger agreement (SHA-256: 7A8C23BE8F24), but incurred high client write latency (2496 ms) because the primary had to wait for the high-latency WAN replica.
2. **Eventual Consistency (W = 1)** reduced write latency by over 99.8% down to 5 ms, but produced clear stale read anomalies on lagging replicas during in-flight replication. We verified that given a quiet period (~3000 ms), the asynchronous queues fully drained and all replicas achieved convergence (SHA-256: 719A65EE7038).
3. **Quorum Consistency (W = 2, R = 2)** struck an optimal balance: by satisfying R + W = 4 > 3, writes completed as soon as the fastest replica acknowledged (1012 ms), while quorum reads successfully detected and discarded stale versions, guaranteeing fresh read linearizability.
4. **Sequence Buffering** successfully mitigated out-of-order network delivery by holding dependent packets and preserving strict causal execution order.
5. **Durability Failure Testing** demonstrated that asynchronous eventual consistency compromises durability under sudden crash failures, resulting in lost loan applications (HL1004, HL1005) during failover.

Architectural Takeaway: For core financial systems like home loan disbursements and EMI repayments, Quorum Consistency (W=2, R=2) or Strong Consistency (W=3) is mandatory to protect data integrity and avoid financial loss, whereas Eventual Consistency should be reserved for non-critical query telemetry or notification feeds.

Attach research paper based on experiment performed into the classroom (Not for all the practicals)

(Students are required to prepare one doc file for each experiment and then upload it in pdf form.)